

Python (as far as we need)

[Welcome to Python.org](#), [3.11.2 Documentation](#), [The Python Standard Library – Python 3.11.2 documentation](#)

Python is a general purpose programming language that supports rapid development of scripts and applications.

Python's main advantages:

- Open Source software, supported by Python Software Foundation
- Available on all platforms
- It is a general-purpose programming language
- Supports multiple programming paradigms
- Very large community with a rich ecosystem of third-party packages

Interpreter

Python is an interpreted language which can be used in two ways:

- “Interactive” Mode: It functions like an “advanced calculator” Executing one command at a time:

```
user:host:~$ python
Python 3.5.1 (default, Oct 23 2015, 18:05:06)
[GCC 4.8.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 2
4
>>> print("Hello World")
Hello World
```

- “Scripting” Mode: Executing a series of “commands” saved in text file, usually with a .py extension after the name of your file:

```
user:host:~$ python my_script.py
Hello World
```

Notebook, or where I have to write the code

[Jupyter Project Documentation](#)

Magics Magics are turning simple python into *magical python*. Magics are the key to power of ipython.

```
# list available python magics
%lsmagic
```

Available line magics:

```
%alias %alias_magic %autocall %automagic %autosave %bookmark %cat %cd %clear %colors %config %connect_
```

Available cell magics:

```
%%! %%HTML %%SVG %%bash %%capture %%debug %%file %%html %%javascript %%js %%latex %%perl %%prun %%py
```

Automagic is ON, % prefix IS NOT needed for line magics.

```
%ll

total 896
-rw-r--r-- 1 dario 35361 Jun 6 08:41 L01Intro.ipynb
-rw-r--r-- 1 dario 259301 Jun 6 15:40 L02IntroMatplotlib.ipynb
-rw-r--r-- 1 dario 22156 Jun 4 11:46 L03fromSensorsToSequence.ipynb
-rw-r--r-- 1 dario 60120 Jun 6 15:54 L04Entrez.ipynb
-rw-r--r-- 1 dario 46866 Jun 6 15:58 L05Parsing.ipynb
-rw-r--r-- 1 dario 97483 Jun 6 16:01 L06Fastq.ipynb
-rw-r--r-- 1 dario 9614 Jun 6 15:22 L07getFastq.ipynb
-rw-r--r-- 1 dario 23110 Jun 6 16:10 L08Blast.ipynb
-rw-r--r-- 1 dario 140775 Jun 5 09:46 L09Alignments.ipynb
```

```
-rw-r--r-- 1 dario 30615 Jun 6 16:17 L10Kegg.ipynb
-rw-r--r-- 1 dario 118039 Jun 4 16:13 L11Cluster.ipynb
-rw-r--r-- 1 dario 8791 Jun 6 09:37 exerciseFastq.fq
drwxr-xr-x 6 dario 204 Jun 6 11:52 images/
-rw-r--r-- 1 dario 29663 Mar 22 2017 jupyterNotebookCheatSheet.pdf
-rw-r--r-- 1 dario 1038 Jun 3 10:36 lib.py
-rw-r--r-- 1 dario 1101 Jun 4 10:14 lib.pyc
drwxr-xr-x 9 dario 306 Jun 3 11:10 maps/
drwxr-xr-x 38 dario 1292 Jun 3 11:10 output/
drwxr-xr-x 13 dario 442 Jun 6 10:06 rawData/
drwxr-xr-x 6 dario 204 Jun 3 11:10 sequences/
```

```
%run
```

```
%run:
```

```
Run the named file inside IPython as a program.
```

```
Usage::
```

```
%run [-n -i -e -G]
      [( -t [-N<N>] | -d [-b<N>] | -p [profile options] )]
      ( -m mod | file ) [args]
```

Parameters after the filename are passed as command-line arguments to the program (put in `sys.argv`). Then, control returns to IPython's prompt.

This is similar to running at a system prompt ``python file args``, but with the advantage of giving you IPython's tracebacks, and of loading all variables into your interactive namespace for further use (unless `-p` is used, see below).

The file is executed in a namespace initially consisting only of `__name__ == '__main__'`` and `sys.argv` constructed as indicated. It thus sees its environment as if it were being run as a stand-alone program (except for sharing global objects such as previously imported modules). But after execution, the IPython interactive namespace gets updated with all variables defined in the program (except for `__name__` and `sys.argv`). This allows for very convenient loading of code for interactive work, while giving each program a 'clean sheet' to run in.

Arguments are expanded using shell-like glob match. Patterns `'*'`, `'?'`, `'[seq]'` and `'[!seq]'` can be used. Additionally, tilde `'~'` will be expanded into user's home directory. Unlike real shells, quotation does not suppress expansions. Use `*two*` back slashes (e.g. ```*```) to suppress expansions. To completely disable these expansions, you can use `-G` flag.

```
Options:
```

```
-n
```

```
__name__ is NOT set to '__main__', but to the running file's name
without extension (as python does under import). This allows running
scripts and reloading the definitions in them without calling code
protected by an ``if __name__ == "__main__"`` clause.
```

```
-i
```

```
run the file in IPython's namespace instead of an empty one. This
is useful if you are experimenting with code written in a text editor
which depends on variables defined interactively.
```

`-e`
ignore `sys.exit()` calls or `SystemExit` exceptions in the script being run. This is particularly useful if IPython is being used to run unittests, which always exit with a `sys.exit()` call. In such cases you are interested in the output of the test results, not in seeing a traceback of the unittest module.

`-t`
print timing information at the end of the run. IPython will give you an estimated CPU time consumption for your script, which under Unix uses the resource module to avoid the wraparound problems of `time.clock()`. Under Unix, an estimate of time spent on system tasks is also given (for Windows platforms this is reported as 0.0).

If `-t` is given, an additional `--N<N>` option can be given, where `<N>` must be an integer indicating how many times you want the script to run. The final timing report will include total and per run results.

For example (testing the script `uniq_stable.py`):

```
In [1]: run -t uniq_stable
```

```
IPython CPU timings (estimated):
```

```
User :    0.19597 s.
```

```
System:    0.0 s.
```

```
In [2]: run -t -N5 uniq_stable
```

```
IPython CPU timings (estimated):
```

```
Total runs performed: 5
```

```
Times :      Total      Per run
```

```
User :   0.910862 s,  0.1821724 s.
```

```
System:    0.0 s,      0.0 s.
```

`-d`
run your program under the control of `pdb`, the Python debugger. This allows you to execute your program step by step, watch variables, etc. Internally, what IPython does is similar to calling:

```
pdb.run('execfile("YOURFILENAME")')
```

with a breakpoint set on line 1 of your file. You can change the line number for this automatic breakpoint to be `<N>` by using the `-bN` option (where `N` must be an integer). For example:

```
%run -d -b40 myscript
```

will set the first breakpoint at line 40 in `myscript.py`. Note that the first breakpoint must be set on a line which actually does something (not a comment or docstring) for it to stop execution.

Or you can specify a breakpoint in a different file:

```
%run -d -b myotherfile.py:20 myscript
```

When the `pdb` debugger starts, you will see a `(Pdb)` prompt. You must first enter `'c'` (without quotes) to start execution up to the first breakpoint.

Entering `'help'` gives information about the use of the debugger. You

can easily see pdb's full documentation with "import pdb;pdb.help()" at a prompt.

-p

run program under the control of the Python profiler module (which prints a detailed report of execution times, function calls, etc).

You can pass other options after -p which affect the behavior of the profiler itself. See the docs for %prun for details.

In this mode, the program's variables do NOT propagate back to the IPython interactive namespace (because they remain in the namespace where the profiler executes them).

Internally this triggers a call to %prun, see its documentation for details on the options available specifically for profiling.

There is one special usage for which the text above doesn't apply: if the filename ends with .ipy[nb], the file is run as ipython script, just as if the commands were written on IPython prompt.

-m

specify module name to load instead of script path. Similar to the -m option for the python interpreter. Use this option last if you want to combine with other %run options. Unlike the python interpreter only source modules are allowed no .pyc or .pyo files. For example::

```
%run -m example
```

will run the example module.

-G

disable shell-like glob expansion of arguments.

```
/Users/dario/anaconda/lib/python2.7/site-packages/IPython/core/magics/execution.py:614: UserWarning: you must provide at least a filename.)
warn('you must provide at least a filename.')
```

```
!dir
```

```
L01Intro.ipynb      L08Blast.ipynb      lib.py
L02IntroMatplotlib.ipynb  L09Alignments.ipynb  lib.pyc
L03fromSensorsToSequence.ipynb  L10Kegg.ipynb      maps
L04Entrez.ipynb      L11Cluster.ipynb      output
L05Parsing.ipynb      exerciseFastq.fq      rawData
L06Fastq.ipynb      images                sequences
L07getFastq.ipynb      jupyterNotebookCheatSheet.pdf
```

Data structures, or where I save my data

simple variables

It is possible to assign values to containers with =.

A container is called **variable** and it has a **name**, a **type** and a **value**

```
aString = 'abcde1234!@'
anInteger = 27
aFloat = 31.0
print (aString)
print (anInteger, aFloat)
```

```
aString
abcde1234!@
(27, 31.0)
```

```
'abcde1234!@'
```

lists

It is possible to collect various data in a **list**.

```
aList = [9, 10.0, 555, 1e-4, 97, 100, 1, 85]
print (aList)

[9, 10.0, 555, 0.0001, 97, 100, 1, 85]
```

A list is defined by means of the square brackets **[]**

You can have a list of numbers, strings, or even lists.

```
aListStrings = ['a', 'ab', 'abc']
aMixedList = [0, 'a', 'a very long string', 1.25e-7, 31]
aListOfLists = [[0, 1, 2], ['a', 'b', 'c']]
```

Something that is handy of the lists is that you can access its elements by means of a number representing a position in the list.

This number is called **index** and in python indexes are **0-based**.

```
aMixedList[2]

'a very long string'
```

It is also possible to extract convenient portion of list by **slicing** them:

```
listName[fromIndex:toIndex]

print (aList[0:2])
print (aList[:2])
print (aList[-2:])
print (aList[::2])
print (aListOfLists[1][0])

[9, 10.0]
[9, 10.0]
[1, 85]
[9, 555, 97, 1]
a
```

It is possible to **extend list** by appending items, the instruction is: `listName.append(newItem)`

```
print (aList[-1:])
aList.append(111)
print (aList[-1:])

[85]
[111]
```

dictionaries

Sometimes it make more sense to refer to data in a tagged manner. On this purpose Python has dictionaries.

A **dictionary** is a set of couples of **keys** and **values**.

A dictionary is defined by means of the brackets **{ }**, and couples of **key: value**

With this structure it is possible to refer to its elements exploiting the keys.

```
aDictionary = {'key1': 9, 'key2': 10.0, 'key3': 555, 'key4': 1e-4}
print (aDictionary['key2'])

10.0
```

Why they are called dictionaries?

```
dictToItalian = {'one': 'uno', 'two': 'due', 'three': 'tre'}
dictToNumbers = {'one': 1, 'two': 2, 'three': 3}
print (dictToItalian['two'], 'is', dictToNumbers['two'])

('due', 'is', 2)

print (dictToNumbers.keys())
print (dictToNumbers.values())

['three', 'two', 'one']
[3, 2, 1]
```

It is possible to **extend dictionaries** by defining new items, the instruction is: dictName[newKey] = value

```
dictToNumbers['four'] = 4
print (dictToNumbers)

{'four': 4, 'three': 3, 'two': 2, 'one': 1}
```

Instructions, or what I can do

arithmetic

```
a = 5
b = 2
x = 5.0
y = 3.0

print( a + b, a * b, a / b, a % b )
print( x / y, x / b, y**b )

(7, 10, 2, 1)
(1.6666666666666667, 2.5, 9.0)
```

comparison

```
print( a > b)

True

print( a == b, a != b)

(False, True)

print( True and True)
print( True and False)
print( True or False)

True
False
True

print( (a > b) and (b != 'a'))

True
```

Control structures, or how I can plan my route

Manipulating data is not enough. You have to do it in the way you mean it, by the right choices and letting the pc do the calculation for you as many times as possible.

To achieve this, you need to control the flow of the information.

indentation

Indentation is the amount of space (4 spaces or multiples of it) inserted at the beginning of a line of code. This mechanism is the way to separate different logic blocks of code.

if-else

This is the simplest way to **choose** how to react when you encounter a bifurcation:

```
if ThisIsTrue :
    doThis
else:
    doThat
```

The if-else control structure is also called selection operator.

Let use a simple example.

Suppose that you need to compute the absolute value of a number stored in a variable x, what you can write is something like:

```
x = -6.29
if x > 0:
    abs = x
else:
    abs = -1 * x
print( abs)

6.29
```

for

This is the way to ask to Python to do the same block of instructions many times.

```
for elements in iterator:
    doThis
    andThis
    andThat
```

The for control structure is also called iteration operator.

Let use a simple example.

Suppose that you need to compute the square value of each number stored in a list, what you can write is something like:

```
theList = [0, 1, 2, 3, 4, 5]
for value in theList:
    print (value**2)

0
1
4
9
16
25

squaresList = []
for value in theList:
    squaresList.append(value**2)
print (squaresList)

[0, 1, 4, 9, 16, 25]
```

You can also store this values in a new list:

Functions, or solve it in simpler problems

A function is both a way to add new command to our language and a leverage to split a complex problem in smaller tasks that are easier to solve.

Functions take **parameters** in input and return data in **output**.

They can be defined as follows:

```
def functionName(parameter1, ....):
    doSomething
    doSomethingMore
    return outputdata
```

A simple example:

Let's start defining the function square

```
def square(aList):
    squaresList = []
    for value in aList:
        squaresList.append(value**2)
    return squaresList
```

now we can use it as we do with any other command of the language

```
print (square(theList))
[0, 1, 4, 9, 16, 25]
```

Libraries, or use what others have done for me

There exists a wide scientific community that uses Python as programming language. Therefore it is possible to exploit their work to simplify our.

This can be done using functions developed by other and distributed in libraries.

It is possible to import a library by means of the command `import`:

```
import os

print( os.getcwd())

/Users/zoipro/Nextcloud/DottoTecSBI/courseMaterial
```

We can exploit the library `os` to make our python code operating system "independent":

```
separator = os.sep
print (separator)

/
```

It is also possible to use nicknames for libraries:

```
import pandas as pd
numbers = pd.Series(theList)
numbers

0    0
1    1
2    2
3    3
4    4
5    5
dtype: int64
```

Dealing with files

To read and write in files in python we need, in first place, to open them.

```
with open(filename, 'how') as stream:
```

where:

- `open` is the command
- `filename` is the path to the file and
- `how` can be one of these options:
 - `r` read mode
 - `w` write mode
- `stream` is the stream to the file.

Once that the stream has been opened we can read and write data with one of the following instructions:

- to write:

```
stream.write(stringToWrite)
```

- to read:

- the whole file
`stringRead = stream.read()`
- a line at a time
`stringLineRead = stream.readline()`

The preferred way to read a big file is to read a line at a time in the following way:

```
with open(filename, 'how') as stream:
    for line in stream:
        doSomething
```

CSV and XLSX files

[pandas - Python Data Analysis Library IO tools \(text, CSV, HDF5, ...\) – pandas documentation](#)

The most convinient way to deal with these two kind of files is to exploit the **pandas** library:

```
import pandas as pd

dfCSV = pd.read_csv(filename)

dfExcel = pd.read_excel(filename)

dfCSV.to_csv(filename)

dfExcel.to_excel(filename)

import pandas as pd
import lib

wrkDirs = lib.setWorkingDirs()
filename = os.path.join(wrkDirs[0], 'MeteoMilano.csv') ## 'rawData/MeteoMilano.csv'

dfMeteo = pd.read_csv(filename)

dfMeteo
```